CHAPITRE VIII

POO: LES EXCEPTIONS

Les Exceptions

Définition

Une exception est un signal indiquant que quelque chose d'exceptionnelle(comme une erreur) s'est produite.
Elle interrompt le flot d'exécution normal du programme

Utilité:

Gérer les erreurs est indispensable : mauvaise gestion peut avoir des conséquences catastrophiques.

Vocabulaire

Lancer ou déclencher (**throw) une exception** consiste à signaler les erreurs.
Capturer ou attraper (**catch) une** exception permet de traiter les erreurs.

Les Exceptions

- Toutes les classes de Java possèdent des exceptions : par exemple, la classe **java.io**, qui gère les entrées - sorties, a, entre autres, l'exception **IOException etc ...**

Exemple de code générant une exception

```
public class CoursExc{
public static void main (String args[])
{
int j = 20, i = 0;
System.out.println(j/i);
System.out.println("coucou c toi !");}}
```

```
public class CoursExc{
public static void main (String args[])
{int[] t=new int[3];
for(int j = 0;j<=3;j++)
{t[j]=j+1;
System.out.print
(t[j]);}
System.out.println("coucou c toi !");}}
```

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
at CoursExc.main(CoursExc.java:5)
)
```

```
123Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 3
at CoursExc.main(CoursExc.java:5)
```

Capturer une exception

- Une exception se capture grâce à un bloc **try {……}**, lorsque le bloc try{...} capture une exception, le traitement associé se trouve dans un bloc **catch {……}.**
- **public class Exception {**
- **public static void main (String args[])**
- {
- **int j = 20, i = 0;**
- **try**{System.*out.println(j/i);}*
- **catch(ArithmeticException e)**
- {
- System.*out.println("Le denominateur ne peut être nul!");*
- }
- System.*out.println(« coucou c'est toi!");*
- }}

Capturer une exception

```
public class Exception {
public static void main (String args[])
{
int j = 20, i = 0;
try
{System.out.println(j/i);}
catch(ArithmeticException e)
{
System.out.println("Le  denominateur ne peut être nul!"+e.getMessage());
}
System.out.println(«  coucou c'est toi!");
}}
```

Lancer ou déclencher une exception

throws Une méthode déclare qu'elle peut lancer une exception par le mot clé throws Ce mot clé est suivi du nom de la classe qui va gérer l'exception.

throw Celui-ci permet d'instancier un objet dans la classe suivant l'instruction throws.

```
• public class CompteBancaire {
•  private double solde;
• public void retrait(double montant) throws SoldeInsuffisant {
• if( solde<montant) throw new SoldeInsuffisant
• (" retrait de" + montant + "impossible");
• solde -= montant;
• }
•  } public class SoldeInsuffisant extends Exception
• { SoldeInsuffisant(String msg) { super(msg); } }
```

TP sur les Exceptions(Lancer et Capturer une exception): Throws,throw

- Créer un classe personne avec une méthode affiche() pour afficher les champs et deux constructeurs l'un sans argument et l'autre avec 3 arguments(Nom,Prenom,age). Mettre une condition dans le constructeur à 3 arguments qui si elle est remplie lance une exception de type AgeNegatifExc.

- Créer une classe TestPer qui permet de tester la classe personne.

  Personne p=new Personne(« BA », «Bintou »,20)

  Je me nomme BA Bintou

  J'ai 20 ans

- Devoir de gérer les erreurs avec un bloc try{} catch{} sinon elles persistent.

TP sur les Exceptions(Lancer et Capturer une exception): Throws, throw, try{ }catch{ }

Capturer ou traiter l'exception au moment de l'instanciation dans la méthode main() et afficher les informations de la personne dans le bloc try.

Personne p=new Personne(« BA », »Bintou »,+/-20)

Deux possibilités:

Je me nomme: BA Bintou

J'ai 20 ans

Ou bien

Rien est exécuté

- Devoir de gérer les erreurs avec un bloc try{} catch{} sinon elles persistent.

TP sur les Exceptions(Lancer et Capturer une exception): Throws, throw, try{ }catch{ }

- Essayer d'afficher les informations à l'extérieur du bloc try{}.
- Que remarquez-vous?
- Essayer de résoudre le problème avec une variable globale.
- Que remarquer vous?
- Essayer de le résoudre avec le constructeur sans paramètres.
- Essayer de gérer deux exceptions à la fois (Exiger plus de 3 caractères pour le prénom par exemple.)
- Fin TP

Finally printStackTrace()

- la méthode **printStackTrace()** permet de voir ou se situe l'exception dans notre code ! Elle vous informe sur le nom de la classe levant l'exception et le numéro de ligne ou se trouve le code l'ayant levée.
- **Finally**: Quoi qu'il se passe dans la classe **try** ou **catch**, les instructions de la clause finally seront TOUJOURS exécutées.
- **Finally**
- {
- System.out.println(« ----------"); System.out.println("Voici le code qui est toujours exécuté ! ! ! ");
- System.out.println("----------\n");
- }

CHAPITRE IX

LES THREADS: (processus)

Définition

- En java, il est possible de simuler l'exécution de plusieurs programmes en même temps . C'est le multithreading. L'exécution de chaque programme est alors appelée un thread.
- Processus vs *threads (= processus légers)*

➢ processus : espaces mémoire séparés

➢ *threads : espace mémoire partagé*

- + efficace
- - robuste
  - • le plantage d'un *thread peut perturber les autres*
  - • le plantage d'un processus n'a pas (normalement) d'incidence sur les autres
- Les threads sont donc des traitements qui vivent au sein d'un même processus.

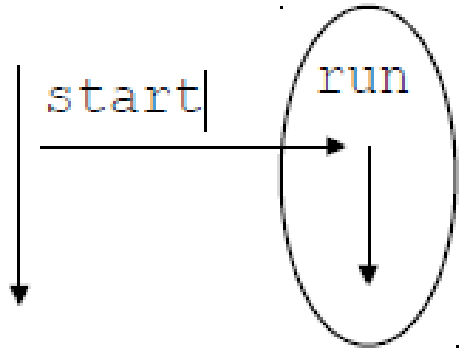
Création de thread

- Plusieurs solutions pour l'écriture d'une classe thread
  - • Classe héritant de java.lang.Thread
  - • Classe implantant l'interface java.lang.Runnable

```
interface Runnable {
        public void run();
}
```

Lancement d'un thread

- Pour lancer un thread on fait appel à la méthode **start().**
- Pour définir ce que fera un thread il faut redéfinir la méthode **run().**



- **Remarques :**
- Création d'autant de threads que nécessaire.
- Un thread meurt lorsque sa méthode **run()** se termine

Lancement d'un thread

- **ATTENTION :**

- L 'appel de la méthode **start()** est fait **une et une seule fois** pour chaque ***thread*.**

- On appelle jamais directement **run(). (start()** le fait**)**

- Le code de la méthode **run()** est exécuté dans le thread courant.

Lancement d’un thread

- En Java le multithreading signifie qu'il faut
  - créer un thread,
  - écrire la tâche exécutée par le thread,
  - lier la tâche au thread.
- Pour réaliser cela, il faut étudier et comprendre la classe Thread.

Constructeur et Méhodes de thread

THREAD	Constructeur(public)	Méthode	Fonct.Méthode
	Thread()	void distroy()	Arrêt brutal du thread
	Thread(Runnable r)	Int getPriority()	Renvoie la priorité du thread
	Thread(Runnable r, String name)	void setPriority(int priority)	Change la priorité du thread
	Thread(String name)	String getName()	Retourne le nom du thread
		Void interrupt()	Pour interrompre le thread
		Static boolean interrupt()	Teste si le thread courant est interrompu
		void join()	Attendre la mort du thread
		Void resume()	Redémarre le thread
		Void run()	Contient le code à

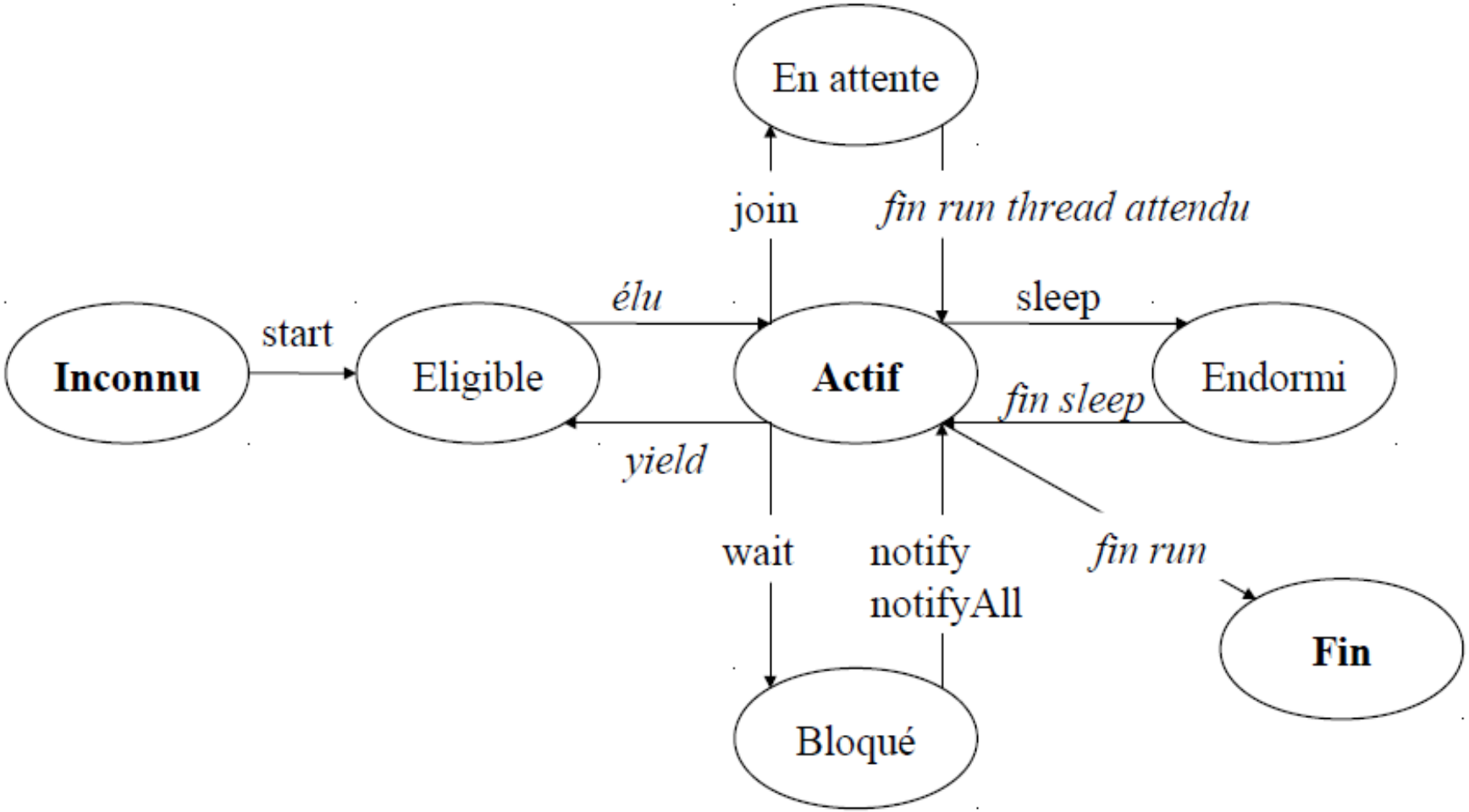
Constructeur et Méhodes de thread

THREAD

Méthode	Fonction de la Méthode
static void sleep(long m)	Mettre en veille le thread
void start()	Démarre l' exécution du thread
String toString()	Renvoie le nom du thread, sa priorité et le groupe auquel il appartient
boolean isAlive()	Indique si le thread est actif ou non
Threadgroup getThreadGroup()	Renvoie un objet qui encapsule le groupe auquel le thread appartient
Void yield()	Indique que le thread peut être suspendu au profits des autres

Cycle de vie d'un thread

- **Résumé du cycle de vie d'un thread:**



Exécuter plusieurs thread simultanément

Héritant de la classe Thread

```
public class MonThread extends Thread {
public void run() {
for (int i = 1; i <= 5; i++)
System.out.println(i + " " + getName());
}
public static void main (String[] args) {
Thread t1=new MonThread();
t1.start();
Thread t2=new MonThread()
t2.start();
}
}
```

```
public class MonThread1 extends Thread {
static int j = 1;
public void run() {
for (int i = 1; i <= 8; i++) {
System.out.println(getName()+" "+j);
j++;}
}
public static void main (String[] args) {
new MonThread1().start();
new MonThread1().start();
}
}
```

```
Thread-1 1
Thread-1 2
Thread-1 3
Thread-0 3
Thread-0 4
Thread-0 5
Thread-0 6
Thread-0 7
Thread-0 8
Thread-0 9
Thread-0 10
Thread-1 12
Thread-1 13
Thread-1 14
Thread-1 15
Thread-1 16
```

Exécuter plusieurs thread simultanément

Implémentant l'interface Runnable

```
public class MonThread2 implements Runnable {
public void run() {
for (int i = 1; i <= 5; i++)
System.out.println(i + " "+Thread.currentThread().getName());
}
public static void main (String[] args) {
Runnable r= new MonThread2();
Thread t1=new Thread(r,"A");
    t1.start();
Thread t2=new Thread(r,"B");
    t2.start();
}
}
```

Exécuter plusieurs thread simultanément

Utilisation de la méthode sleep

```
class A extends Thread {
public void run() {
for(int i=0; i<8; i++) {
System.out.print("A"+i+" ");
try {sleep(100);} catch
(InterruptedException e) {}
}
}

class B extends Thread {
public void run() {
for(int i=0; i<8; i++) {
System.out.print("B"+i+" ");
try {sleep(200);} catch
(InterruptedException e) {}
}
}
```

```
public class TestThread1 {
public static void main(String args[]) {
new A().start(); new B().start();
}
```

Exclusion et moniteurs: Synchronized

- Pour que plusieurs processus puissent se partager sainement des objets afin de coopérer ou d'utiliser des ressources, il faut pouvoir limiter le parallélisme en s 'assurant qu'un certain objet ne subisse pas en même temps plusieurs séquences d'actions :

```
class Impr extends Thread {
String txt;
public Impr(String txt){this.txt=txt;}
public void run() {
for(int j=0; j<2; j++) {
for(int i=0; i<txt.length(); i++) {
try {sleep(100);} catch
(InterruptedException e) {};
System.out.print(txt.charAt(i));
}
}
```

```
public class TestThread3 {
 public void main(String argv[]) {
new Impr("BONJOUR ").start();
new Impr("AU REVOIR ").start();
}
}
```

Exclusion: Synchronized

- Pour assurer les exclusions, Java offre la primitive **synchronized.** En Java *tout objet est susceptible d'être un motif d'exclusion. La syntaxe générale d'une exclusion* relative à un objet obj est :
- synchronized(obj) { *bloc d'instructions }*
- On peut alors programmer comme suit l'exclusion souhaitée :

- class Exclusion {};

Exclusion: Synchronized

```
class ImpSync  extends Thread {
String txt;
 Exclusion excImp = new Exclusion();
public ImpSync(String txt){this.txt=txt;}
public void run() {
for(int j=0; j<2; j++) {
synchronized(excImp){
for(int i=0; i<txt.length(); i++) {
try {sleep(100);} catch(InterruptedException e) {}
System.out.print(txt.charAt(i));
}
System.out.print(" ");
}}}}
```

Moniteurs: Synchronized

- L'exclusion peut porter sur une méthode complète. La syntaxe est dans ce cas :

```
class A {
 ...
synchronized ... P(...) { ... }
 ...
 }
```

```
class MoniteurImpression {
synchronized void imprTexte(String txt) {
for(int i=0; i<txt.length(); i++) {
try {Thread.sleep(100);}
catch(InterruptedException e) {}
System.out.print(txt.charAt(i));
}
}
}
```

Moniteurs: Synchronized

```
class ImprMonit extends Thread {
String txt;
 MoniteurImpression m1 = new MoniteurImpression();
public ImprMonit(String t){txt=t;}
public void run() {
for(int j=0; j<2; j++) { m1.imprTexte(txt);};
}
}
```

*Attente explicite : wait - notify

- Attente qu'une condition soit satisfaite : **wait()**
- À l'intérieur d'un moniteur (c'est à dire d'une méthode ou d'un bloc qualifié par synchronized), un processus peut se mettre en attente au moyen de **wait().**
- Cette primitive est susceptible de déclencher, pendant l'attente, une exception de type **InterruptedException,** ce qui oblige souvent à utiliser la forme :

```
try {wait();} catch(InterruptedException e) {...}
```

- Réveil des processus : **notify() et notifyAll()**
- Un processus sort de l'attente lorsqu'un autre processus exécute notify() au sein de ce même moniteur. Cette primitive existe sous deux formes :
- **notify() :** relance *un processus en attente dans ce moniteur,*
- **notifyAll():** relance *tous les processus en attente dans ce moniteur.*

*Attente explicite : wait - notify

- L'exemple suivant illustre la programmation du classique tampon producteur-consommateur.
- class MoniteurProdCons {
- String tampon; boolean estVide=true;
- synchronized void prod(String m) {
- if(!estVide){ System.out.println("PRODUCTEUR ATTEND");
- try {wait();} catch(InterruptedException e) {};
- }
- System.out.println("PRODUIT : " + m);
- tampon=m; estVide=false; notify();
- }
- synchronized String cons() {
- if(estVide){ System.out.println("CONSOMMATEUR ATTEND");
- try {wait();} catch(InterruptedException e) {};
- }
- System.out.println("CONSOMME : " + tampon);
- String resul=tampon; estVide=true; notify();
- return resul;
- }
- }

*Attente explicite : wait - notify

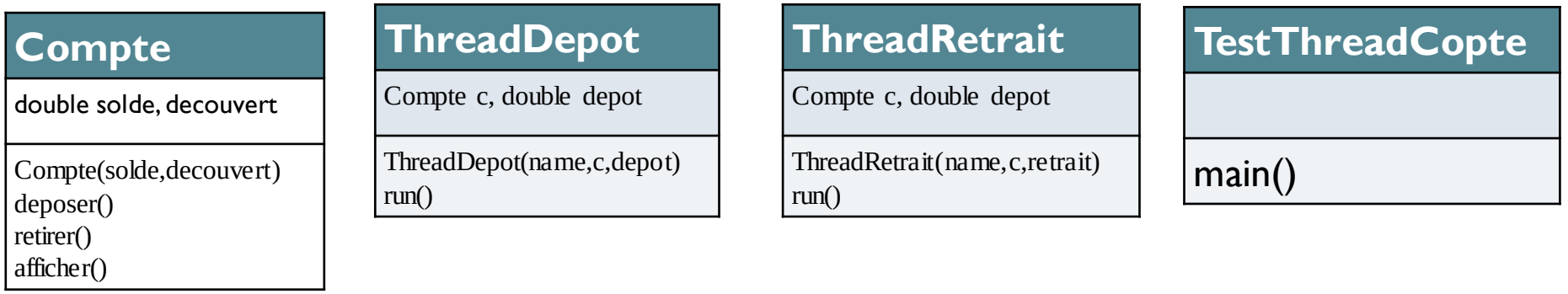
```
class Producteur extends Thread {
MoniteurProdCons tampon;
public Producteur (MoniteurProdCons tampon) { this.tampon=tampon;}
public void run() {
tampon.prod("message1"); tampon.prod("message2");
try {sleep(100);} catch(InterruptedException e) {};
tampon.prod("message3");
}}
class Consommateur extends Thread {
MoniteurProdCons tampon;
public Consommateur(MoniteurProdCons tampon) { this.tampon=tampon;}
public void run() {
tampon.cons(); tampon.cons(); tampon.cons();
}}
```

*Attente explicite : wait - notify

- Le programme principal crée un tampon et une paire de processus producteur/consommateur
- qui communiquent par ce tampon :
- public class TestWait {
- public static void main(String argv[]) {
- MoniteurProdCons tampon = new MoniteurProdCons();
- new Producteur(tampon).start();
- new Consommateur(tampon).start();
- }}

*TRAVAUX PRATIQUES

- Considérons une classe Compte permettant de gérer les comptes de clients dans une quelconque banque disposant de plusieurs agences.
- Pour une gestion efficace et sécurisée des comptes client, il ne faudrait pas permettre par exemple que deux (au moins) transactions s'effectuent simultanément sur un même compte à un instant donné (exemple: le retrait et le dépôt).
- *Les opérations de retrait et de dépôt sont gérées séparément par deux threads distincts.*
- *Votre travail sera de créer ces deux threads de telle sorte qu'ils ne pourront jamais s'exécuter simultanément.*
- *Pour cela créez les classes suivantes:*



CHAPITRE X

POO: LES ENTREES/SORTIES: (FLUX)

Présentation des FLux

- Les flux (stream en anglais) permettent d'encapsuler des processus d'envoi et de réception de données. Les flux traitent toujours les données de façon séquentielle.
- En java, les flux peuvent être divisés en plusieurs catégories :
  - les flux d'entrée (input stream) et les flux de sortie (output stream).
  - les flux de traitement de caractères et les flux de traitement d'octets.
- Java définit des flux pour lire ou écrire des données mais aussi des classes qui permettent de faire des traitements sur les données du flux. Ces classes doivent être associées à un flux de lecture ou d'écriture et sont considérées comme des filtres. Par exemple, il existe des filtres qui permettent de mettre les données traitées dans un tampon (buffer) pour les traiter par lots.

Toutes ces classes sont regroupées dans le package java.io.

Les Classes de gestion des FLux

- L'utilisation de ces classes n'est pas chose facile, vu leur nombre élevé et la difficulté de choisir la classe convenable à un besoin précis.
- Pour bien choisir une classe adaptée à un traitement donné, il faut comprendre la dénomination des différentes classes.
- Le nom d'une classe est toujours composé d'un **préfixe** et d'un **suffixe**.. Il y a quatre suffixes possibles en fonction du type de flux (flux d'octets ou de caractères) et du sens du flux (entrée ou sortie).

	Flux d'octets	Flux de caractères
Flux d'entrées	InputStream	Reader
Flux de sorties	OutputStream	Writer

Les Classes de gestion des FLux

Il existe donc quatre hiérarchies de classes qui encapsulent des flux particuliers.
Ces classes peuvent être séparées en deux séries de deux catégories différentes: les classes de lecture et d'écriture et les classes de manipulation de caractères et d'octets.

- ✓ les sous classes de **Reader** sont des types de flux en **lecture** sur les **caractères,**
- ✓ les sous classes de **Writer** sont des types de flux en **écriture** sur les **caractères,**
- ✓ les sous classes de **InputStream** sont des types de flux en **lecture** sur les **octets,**
- ✓ les sous classe de **OutputStream** sont des types de flux en **écriture** sur les **octets.**

Les Classes de gestion des FLux

On peut distinguer un ensemble de préfixes qui contiennent le type de traitement qu'il effectue.

Buffered : ce type de filtre permet de mettre les données du flux dans un tampon. Il peut être utilisé en entrée et en sortie.
Sequence : ce filtre permet de fusionner plusieurs flux.
Data : ce type de flux permet de traiter les octets sous forme de type de données.
LineNumber : ce filtre permet de numéroter les lignes contenues dans le flux.
PushBack : ce filtre permet de remettre des données lues dans le flux.
Print : ce filtre permet de réaliser des impressions formatées.
Object : ce filtre est utilisé par la sérialisation
InputStream / OuputStream : ce filtre permet de convertir des octets en caractères.

Buffered
Sequence
Data
LineNumber
PushBack
Print
Object
OutputStream

Les Flux de caractères en lecture/écriture

Ils transportent des données sous forme de caractères : java les gèrent avec le format Unicode qui code les caractères sur 2 octets. Les classes qui gèrent les flux de caractères héritent d'une des deux classes abstraites Reader ou Writer. Il existe de nombreuses sous classes pour traiter les flux de caractères.

	Flux en lecture	Flux en sortie
Flux de caractères	BufferedReader CharArrayReader FileReader InputStreamReader LineNumberReader PipedReader PushbackReader StringReader	BufferedWriter CharArrayWriter FileWriter OutputStreamWriter PipedWriter StringWriter

La classe Reader

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en lecture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
boolean markSupported()	indique si le flux supporte la possibilité de marquer des positions
boolean ready()	indique si le flux est prêt à être lu
close()	ferme le flux et libère les ressources qui lui étaient associées
int read()	renvoie le caractère lu ou -1 si la fin du flux est atteinte.
int read(char[])	lire plusieurs caractères et les mettre dans un tableau de caractères
int read(char[], int, int)	lire plusieurs caractères. Elle attend en paramètre : un tableau de caractères qui contiendra les caractères lus, l'indice du premier élément du tableau qui recevra le premier caractère et le nombre de caractères à lire. Elle renvoie le nombre de caractères lus ou -1 si aucun caractère n'a été lu. Le tableau de caractères contient les caractères lus.

La classe FileReader

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Reader.

Constructeur	Rôle
FileInputReader(String)	Créer un flux en lecture vers le fichier dont le nom est précisé en paramètre.
FileInputReader(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileReader fichier = new FileReader("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileReader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Reader et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

La classe Writer

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en écriture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
close()	ferme le flux et libère les ressources qui lui étaient associées
write(int)	écrire le caractère en paramètre dans le flux.
write(char[])	écrire le tableau de caractères en paramètre dans le flux.
write(char[], int, int)	écrire plusieurs caractères. Elle attend en paramètres : un tableau de caractères, l'indice du premier caractère dans le tableau à écrire et le nombre de caractères à écrire.
write(String)	écrire la chaîne de caractères en paramètre dans le flux
write(String, int, int)	écrire une portion d'une chaîne de caractères. Elle attend en paramètre : une chaîne de caractères, l'indice du premier caractère dans la chaîne à écrire et le nombre de caractères à écrire.

La classe FileWriter

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Writer.

Constructeur	Rôle
FileWriter(String)	Si le nom du fichier précisé n'existe pas alors le fichier sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileWriter(File)	Idem mais le fichier est précisé avec un objet de la classe File.
FileWriter(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileWriter fichier = new FileWriter("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileWriter qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Writer et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

Les Flux de caractères tamponnés:

Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble de caractères représentant une ligne plutôt que de traiter les données caractères par caractères. Le nombre d'opérations est ainsi réduit.
Les classes **BufferedReader** et **BufferedWriter** permettent de gérer des flux de caractères tamponnés avec des fichiers.

La classe BufferedReader

Cette classe possède plusieurs constructeurs qui peuvent tous lever une exception de type FileNotFoundException:

Constructeur	Rôle
BufferedReader(Reader)	le paramètre fourni doit correspondre au flux à lire.
BufferedReader(Reader, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception de type IllegalArgumentException est levée.

```
BufferedReader fichier = new BufferedReader(new FileReader("fichier.txt"));
```

La classe BufferedReader

Il existe plusieurs méthodes de la classe BufferedReader héritées de la classe Reader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes peuvent lever une exception de type IOException. Elle définit une méthode supplémentaire pour la lecture :

Méthode	Rôle
String readLine()	lire une ligne de caractères dans le flux. Une ligne est une suite de caractères qui se termine par un retour chariot '\r' ou un saut de ligne '\n' ou les deux.

La classe BufferedReader possède plusieurs méthodes pour gérer le flux hérité de la classe Reader.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple de lecture d'un fichier texte

```
import java.io.*; public class TestBufferedReader {

public void lecture() {
 try {
String ligne ;
   FileReader fr=new FileReader(source);
BufferedReader  br = new BufferedReader(fr);
while ((ligne  = br.readLine()) != null)
{ System.out.println(ligne);  }
 br.close();
fr.close(); }
 catch (Exception e)
 { e.printStackTrace(); }} }

Public class TestBR
public static void main(String  args[]) {
 TestBufferedReader tb=new TestBufferedReader();
tb.lecture(); }
```

La classe BufferedWriter

Cette classe possède plusieurs constructeurs Exemple:

Constructeur	Rôle
BufferedWriter(Writer)	le paramètre fourni doit correspondre au flux dans lequel les données sont écrites.
BufferedWriter(Writer, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception IllegalArgumentException est levée.

```
BufferedWriter fichier = new BufferedWriter( new FileWriter("monfichier.txt"));
```

Exemple d'écriture dans un fichier texte

```
import java.io.*; import java.util.*;
 public class TestBufferedWriter
{
private void traitement() {
try { String ligne ; int nombre = 123;
FileWriter fw=new FileWriter(destination);
BufferedWriter bw = new BufferedWriter(fw);
bw.write("bonjour tout le monde");
bw.newLine();
bw.write("Nous sommes le "+ new Date());
bw.write(", le nombre magique est " + nombre);
 fw.close(); } catch (Exception e)
 { e.printStackTrace(); } } }


Public class TestBW
public static void main(String args[])
 {TestBufferedWriter tbw=new TestBufferedWriter();
            tbw.traitement(); }
```

La classe PrintWriter

Cette classe permet d'écrire dans un flux des données formatées.
Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
PrintWriter(Writer)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(Writer, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé
PrintWriter(OutputStream)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(OutputStream, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé

La classe PrintWriter

```
PrintWriter fichier = new PrintWriter( new FileWriter("monfichier.txt"));
```

Il existe de nombreuses méthodes de la classe PrintWriter qui permettent d'écrire un ou plusieurs caractères dans le flux en les formatant. Les méthodes write() sont héritées de la classe Writer. Elle définit plusieurs méthodes pour envoyer des données formatées dans le flux :

print(...)

Plusieurs méthodes print acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux

println()

Cette méthode permet de terminer la ligne courante dans le flux en y écrivant un saut de ligne.

println (...)

Plusieurs méthodes println acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux avec une fin de ligne.

La classe PrintWriter possède des méthodes pour gérer le flux :

Méthode	Rôle
flush()	Vide le tampon en écrivant les données dans le flux.

Exemples

```
import java.io.*;
 import java.util.*;
public class TestPrintWriter {
private void traitement() {
try { String ligne ; int nombre = 123;
PrintWriter fichier = new PrintWriter(new FileWriter(destination));
fichier.println("bonjour tout le monde");
fichier.println("Nous sommes le "+ new Date());
fichier.println("le nombre magique est " + nombre);
fichier.close(); } catch (Exception e)
{ e.printStackTrace(); } }

public static void main(String args[]) {
TestPrintWriter  tp=new TestPrintWriter();  tp.traitement();}
}
```

Les Flux d’octets

Ils transportent des données sous forme d'octets. Les flux de ce type sont capables de traiter toutes les données.
Les classes qui gèrent les flux d'octets héritent d'une des deux classes abstraites InputStream ou OutputStream. Il existe de nombreuses sous classes pour traiter les flux d'octets.

Les flux d'octets avec un fichier.

Les classes FileInputStream et FileOutputStream permettent de gérer des flux d'octets avec des fichiers.

Les flux d'octets en lecture sur un fichier

Il faut instancier un objet de la classe FileInputStream. Cette classe possède plusieurs constructeurs qui peuvent tous lever l'exception FileNotFoundException:

Constructeur	Rôle
FileInputStream(String)	Ouvre un flux en lecture sur le fichier dont le nom est donné en paramètre
FileInputStream(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileInputStream fichier = new FileInputStream("monfichier.dat");
```

Les flux d'octets en lecture sur un fichier

Il existe des méthodes de la classe FileInputStream qui permettent de lire un ou plusieurs octets dans le flux. Toutes ces méthodes peuvent lever l'exception IOException.

int read( )

Cette méthode envoie la valeur de l'octet lu ou -1 si la fin du flux est atteinte.

int read(byte[], int, int), int read(byte[ ])

Cette méthode lit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contiendra les octets lus, l'indice du premier élément du tableau qui recevra le premier octet et le nombre d'octets à lire.

Elle renvoie le nombre d'octets lus ou -1 si aucun octet n'a été lu. Le tableau d'octets contient les octets lus.

Les flux d'octets en lecture sur un fichier

La classe FileInputStream possède plusieurs méthodes pour gérer le flux :

Méthode	Rôle
long skip(long)	saute autant d'octets dans le flux que la valeur fournie en paramètre. Elle renvoie le nombre d'octets sautés.
close()	ferme le flux et libère les ressources qui lui étaient associées
int available()	retourne une estimation du nombre d'octets qu'il est encore possible de lire dans le flux

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Les flux d'octets en écriture sur un fichier

Il faut instancier un objet de la classe FileOutputStream. Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
FileOutputStream(String)	Si le fichier précisé n'existe pas, il sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileOutputStream(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileOuputStream fichier = new FileOutputStream("monfichier.dat");
```

Les flux d'octets en écriture sur un fichier

Il existe plusieurs méthodes de la classe FileOutputStream qui permettent de lire un ou plusieurs octets dans le flux.

write(int)

Cette méthode écrit l'octet en paramètre dans le flux.

write(byte[])

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire : tous les éléments du tableau sont écrits.

write(byte[], int, int)

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire, l'indice du premier élément du tableau d'octets à écrire et le nombre d'octets à écrire.

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class CopieFichier {
public static void main(String args[]) {
try { copierFichier("source.txt", "copie.txt"); }
catch (IOException e) { e.printStackTrace(); } }
 public static void copierFichier(String source, String destination) throws
IOException {
FileInputStream fis = null;
FileOutputStream fos = null; try { byte buffer[] = new byte[1024];
int taille = 0;
fis = new FileInputStream(source);
fos = new FileOutputStream(destination);
while ((taille = fis.read(buffer)) != -1)
{ System.out.println(taille);
fos.write(buffer, 0, taille); } }
 finally { if (fis != null) { try { fis.close(); } catch (IOException e) { } } if
(fos != null) { try { fos.close(); } catch (IOException e) { } } } }
```

La classe File

Une instance de la classe File est une représentation logique d'un fichier ou d'un répertoire qui peut ne pas exister physiquement sur le disque.
Si le fichier ou le répertoire existe, de nombreuses méthodes de la classe File permettent d'obtenir des informations sur le fichier. Sinon plusieurs méthodes permettent de créer des fichiers ou des répertoires. Voici une liste de quelques méthodes :

Méthode	Rôle
boolean canRead()	indique si le fichier peut être lu
boolean canWrite()	indique si le fichier peut être modifié
boolean exists()	indique si le fichier existe physiquement
String getAbsolutePath()	renvoie le chemin absolu du fichier
String getPath	renvoie le chemin du fichier
boolean isDirectory()	indique si le fichier est un répertoire
long length()	renvoie la longueur du fichier
String[] list()	renvoie la liste des fichiers et répertoire contenu dans le rép
boolean mkdir()	création du répertoire
boolean mkdirs()	création du répertoire avec création des répertoires manquants dans l'arborescence du chemin

Utilisation de quelques méthodes

```
import java.io.*;
 public class TestFile {
 protected String nomFichier;
protected File fichier;
 public TestFile(String nomFichier) { this.nomFichier = nomFichier;
 fichier = new File(nomFichier);
 traitement(); }
private void traitement() {
 if (!fichier.exists()) {
 System.out.println("le fichier "+nomFichier+"n'existe pas");
 System.exit(1); }
 System.out.println(" Nom du fichier : "+fichier.getName());
 System.out.println(" Chemin du fichier : "+fichier.getPath());
 System.out.println(" Chemin absolu : "+fichier.getAbsolutePath());
 System.out.println(" Droit de lecture : "+fichier.canRead());
 System.out.println(" Droit d'ecriture : "+fichier.canWrite());
 if (fichier.isDirectory())
 { System.out.println(" contenu du repertoire ");
String fichiers[] = fichier.list();
for(int i = 0; i < fichiers.length; i++)
System.out.println(" "+fichiers[i]); } } }
```

```
 public class TestMethodesFile
public static void main(String args[]) {
 new TestFile(args[0]); }
```

Les fichiers à accès direct

Les fichiers à accès direct permettent un accès rapide à un enregistrement contenu dans un fichier.

La classe RamdonAccessFile encapsule les opérations de lecture/écriture d'un tel fichier.
Elle implémente les interfaces **DataInput** et **DataOutput.**
Elle possède deux constructeurs qui attendent en paramètre **le fichier à utiliser** (sous la forme d'un nom de fichier ou d'un objet de type File qui encapsule le fichier) et **le mode d'accès**.
Le mode est une chaîne de caractères qui doit être égal à **«r»** ou **«rw»** selon que le mode est lecture seule ou lecture/écriture.
Ces deux constructeurs peuvent lever les exceptions suivantes :
FileNotFoundException si le fichier n'est pas trouvé
IllegalArgumentException si le mode n'est pas «r» ou «rw»
SecurityException si le gestionnaire de sécurité empêche l'accès aux fichiers dans le mode précisé

Les fichiers à accès direct

La classe RandomAccessFile possède de nombreuses méthodes writeXXX() pour écrire des types primitifs dans le fichier

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw");
for (int i = 0; i < 10; i++)
{ monFichier.writeInt(i * 100); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

Les fichiers à accès direct

Elle possède aussi de nombreuses classes readXXX() pour lire des données primitives dans le fichier.

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw"); for (int i =
0; i < 10; i++)
{ System.out.println(monFichier.readInt()); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

CHAPITRE XI

GUI: LES INTERFACES GRAPHIQUES (JavaFx)

HISTORIQUE(Chronologie)

1995	1996	1998	2007	2011	2014-2018
AWT	JAVA1.0 SWING	JAVA1.2	Annonce de JavaFX	Java7	Java8-JavaFX

A l'origine du langage Java, les interfaces graphiques étaient créées en utilisant la librairie AWT (java.awt)

Rapidement, la librairie Swing (javax.swing) est venu compléter (et partiellement remplacer) la librairie AWT

HISTORIQUE(SWING vs AWT)

- Tous les composants du package AWT sont des composants lourds.
- Les composants Swing (nom du package : javax.swing) sont pour la majorité d'entre eux des composants légers.
- Tous les composants Swing commencent par la lettre " J ".
- Les composants Swing se répartissent :
  - en conteneurs de plus lourd:
    - **JFrame à rapprocher de la classe Frame dans AWT**
    - **JDialog à rapprocher de la classe Dialog dans AWT**
    - **JWindow à rapprocher de la classe Window dans AWT**
    - **JApplet à rapprocher de la classe Applet dans AWT**
  - en conteneurs légers (les autres contrôles GUI Swing).

HISTORIQUE(SWING vs AWT)

JavaFX 1	JavaFX 2	JavaFX 8
a tenté, sans grand succès, de remplacer Swing	Pas de script. Java API et/ou XML:FXML SceneBuilder, CSS	Plus de librairie externe à télécharger et à référencer **Scene Builder 2 , lambda** et les streams ,DatePicker TreeTableView. Gestion des écrans tactiles (TouchEvent,GestureEvent,) Amélioration des librairies graphiques 2D et 3D ,CSS

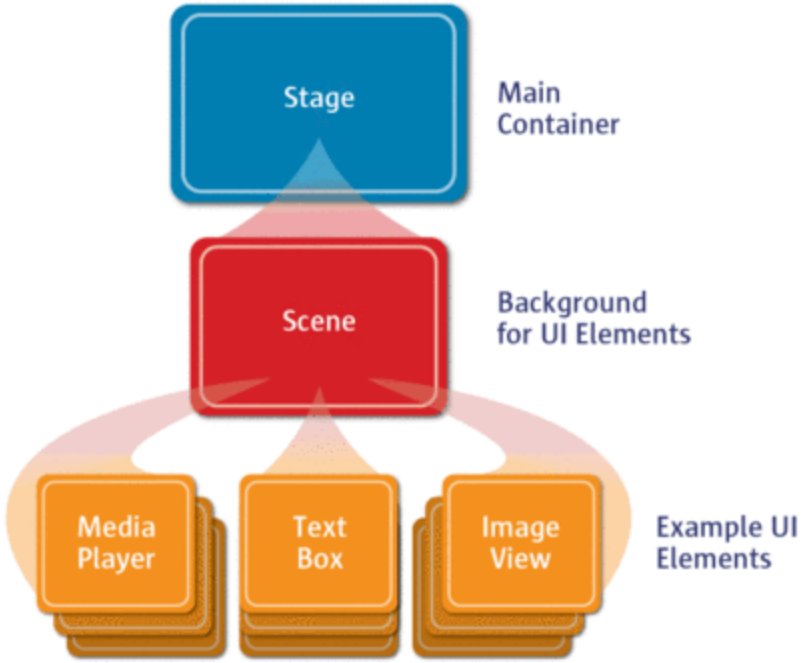
Presentation d'une application JavaFX

Pour construire une application JavaFX il faut:

- ➢Hériter de la classe Application.

- ➢Créer un objet de type **Stage** pour le lancement de l'application.

- ➢Créer un objet de type **Scene** et l'associer à la fenêtre(Stage).

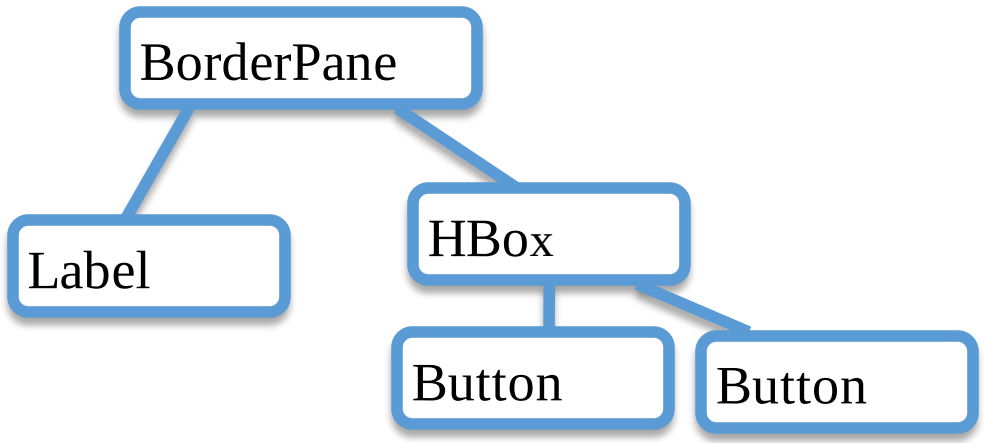


- ➢La Scène est composée de nœuds (**Node**)qui représente les différents éléments de l'interface graphique.

- ➢Enfin on appel la méthode start() qui construit l'interface.

Graphe de Scene

Forme primitive (**Shape**)	Conteneurs(**Layout-Pane**)	Composant standard (**Control**)	Composants spécialisés
Line, Circle,Rectangle, Box, Cylinder	AnchorPane, StackPane, BorderPane, GridPane, HBox, Vbox,	Label, Button, TetField, ComboBox	MediaView, WebView, ImageView,



Creation d'une fenetre personnalisee

```
public class PremiereFenetre extends Application {
Button bouton;
public void start(Stage premierStage)
{
    premierStage.setTitle("Ma premiere application javaFx");
    bouton=new Button();
    bouton.setText("Click me");
    StackPane root=new StackPane();
    root.getChildren().add(bouton);
    Scene scene=new Scene(root,300,250);
    premierStage.setScene(scene);
    premierStage.show();
}
public static void main(String[] args) {
launch(args);}}
```

Action sur un Bouton

```
public class PremiereFenetre extends Application
implements EventHandler<ActionEvent>{
Button bouton;
public void start(Stage premierStage)
{
        premierStage.setTitle("Ma  premiere application javaFx");
        bouton=new Button();
        bouton.setText("Click me");
        StackPane root=new StackPane();
        root.getChildren().add(bouton);
        Scene scene=new Scene(root,300,250);
        bouton.setOnAction(this);
        premierStage.setScene(scene);
        premierStage.show();}
        public static void main(String[] args) {
        launch(args);}
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous  m'avez touche");}}
```

La méthode **gerSource** fournit une référence sur l'objet ayant déclenché l'événement concerné.

Classe Anonyme

Il est possible de définir une classe interne, sans lui donner de nom par dérivation d'une super classe, ou par implémentation d'une interface.

```
bouton.setOnAction(new
EventHandler<ActionEvent>(){
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous m'avez touche"); }});
```

Les expressions Lamda

```
bouton.setOnAction(e->System.out.println("Vous m'avez touche"));
```

Les expressions **lambd**a sont aussi nommées **closures** ou fonctions anonymes : leur but principal est de permettre de passer en paramètre un ensemble de traitements.

Les événements clavier

- **onKeyPressed:** touche enfoncée
- **onKeyReleased:** touche relâchée
- **onKeyTyped:** touche saisie

Pour recevoir des événements il faut obtenir le focus. On doit mettre à true le focusTransversable

Les événements clavier

```
public class EvenementClavierSouris extends Application{
//Button bouton;
public void start(Stage premierStage)
{
Circle cercle=new Circle();
cercle.setCenterX(100);
cercle.setCenterY(100);
cercle.setRadius(50);
cercle.setFill(Color.AQUAMARINE);
cercle.setFocusTraversable(true);
cercle.setOnKeyPressed(KeyEvent->System.out.prinln("Touche enfoncee: :"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyReleased(KeyEvent->System.out.println("Touche relachee:"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyTyped(KeyEvent->System.out.println("touche saisie: "+KeyEvent.getCode()+ " "
+KeyEvent.getCharacter()));
premierStage.setTitle("Evenement clavier");
Pane root=new Pane();
root.getChildren().setAll(cercle);
Scene scene=new Scene(root,300,250,Color.GREEN);
premierStage.setScene(scene);
premierStage.show();}
public static void main(String[] args) {
launch(args);}}
```

Les événements souris

- **onMouseClicked:** clic de souris
- **onMouseDragged:** la souris cliquée-tiree
- **onMouseEntered:** la souris est entrée dans la zone du nœud
- **onMouseExited:** la souris est sortie de la zone
- **onMouseMoved:** la souris est deplacee
- **onMousePressed:** la souris est enfoncée
- **onMousereleased:** la souris est relâchée

On n'a pas besoin de mettre à true le focusTransversable pour obtenir le focus.

Les événements souris

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez clique sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseDragged(MouseEvent->System.***out.println("Vous avez effectue un drag'n drop sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseEntered(MouseEvent->System.***out.println("Vous avez entre la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseExited(MouseEvent->System.***out.println("Vous avez sorti la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseMoved(MouseEvent->System.***out.println("Vous avez deplace sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMousePressed(MouseEvent->System.***out.println("Vous avez appuye la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez relache la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

Naviguer entre deux scenes

```
public class Mafenetre extends Application
{Stage window;
Scene scene1,scene2;
public static void main(String args[])
{launch(args);}
public void start(Stage primaryStage) throws Exception {
//****** Initialisation et instanciation ***************
window=primaryStage;
Label label = new Label("Bienvenue sur la premiere Scene");
Button bouton1 = new Button("premiere Scene");
Button bouton2 = new Button("Deuxieme Scene");
VBox vb=new VBox(20);
StackPane stp= new StackPane();
scene1=new Scene(vb,300,200);
//******************** Ajout de composant ***************************
vb.getChildren().addAll(bouton1,label);
stp.getChildren().add(bouton2);
scene2=new Scene(stp,500,300,Color.RED);
window.setScene(scene2);
window.setTitle("Naviger");
//******************** Action ***************************
bouton1.setOnAction(e->window.setScene(scene2));
bouton2.setOnAction(e->window.setScene(scene1));
window.show();}}
```

Label: Pour afficher du texte dans un conteneur.

VBox est un conteneur qui arrange les sous-composants sur une seule colonne

Layout-Panes

Les layout permettent aux utilisateurs d'avoir un large choix sur la structure et la position qu'il faut adopter pour les composants qui seront placés sur la fenêtre.

Hbox-VBox	Les composants sont placés horizontalement ou verticalement.
FlowPane(Horizontal-Vertical)	Les composants sont placés horizontalement/verticalement avant de passer à la ligne/colonne suivante s'il n'ya plus d'espace.
BorderPane	Les composants peuvent être placés dans cinq emplacements différents: Top, Bottom, Left, Right, Center.
AnchorPane	Place les composants en respectant une contrainte de distance par rapport à un ou plusieurs bords du conteneur.
StackPane	Place les composants les uns au dessus des autres
GridPane	Place les composants dans une grille

Couleur de bordures et d'arrière-plans

```
Background bgGreen = new Background( new BackgroundFill(Color.LIGHTGREEN,
new CornerRadii(20), new Insets(15) ));
hb.setBackground(bgGreen);
```

```
Background bgImg = new Background( new BackgroundImage( img,null,null,
BackgroundPosition.CENTER, null));
hb.setBackground(bgImg);
```

Les composants(Les etiquettes et les champs de texte)

Pour afficher du texte dans un conteneur on utilise la classe **Label:**

Label lb=new Label("Factoriel");

Le texte d'une étiquette peut être modifier avec la méthode **setText()**

Un champ de texte est une zone rectangulaire dans laquelle l'utilisateur peut entrer ou modifier des valeurs. On utilise la classe **TextField** pour instancier un tel objet.

```
TextField tf=new TetField();
tf.setPromptText (" valeur à calculer ");// valeur par défaut
```

L'information figurant dans un champs texte s'obtient avec la méthode **getText().**

PasswordField permet de saisir un texte sans que celui-ci soit affiché dans le champ texte.

Pour utiliser une zone de texte multilignes on utilise la classe **TextArea.**

```
TextArea txa = new TextArea();
txa. setPrefRowCount(8);
```

Les composants(Bouton radio)

On utilise la classe **RadioButton** pour les boutons radio.

```
ToggleGroup tg = new ToggleGroup();
 on.setToggleGroup(tg);
off.setToggleGroup(tg);
```

Les actions sur les boutons radio:

```
tg.selectedToggleProperty().addListener(new ChangeListener<Toggle>() {
        @Override
        public void changed(ObservableValue<? extends Toggle> ov, Toggle old_toggle,
Toggle new_toggle) {

            if (tg.getSelectedToggle() != null) {
                RadioButton button = (RadioButton) tg.getSelectedToggle();
                System.out.println("Bouton:  " + button.getText());
                }} });
```

Les composants(les cases à cocher)

- Les cases à cocher permettent à l'utilisateur d'effectuer un choix de type oui/non. Elle est instancier grâce à la classe CheckBox.
- CheckBox cb=new CheckBox();

Les actions sur les cases à cocher:

```
CheckBox cb=new CheckBox(« marié");
   cb.setOnAction((event) -> {
  if(cb.isSelected())
   System.out.println("la case "+cb.getText()+" est selectionnée");
});
```

Les composants(Hyperlink)

- Hyperlink est un composant semblable à un lien hypertexte HTML dans une page web.

```
Hyperlink hl=new Hyperlink("Site de l'UADB");
   hl.setOnAction(event-> { getHostServices().showDocument("www.uadb.edu.sn"); });
```

Tooltip peut être ajouté à n'importe quel composant:

```
Tooltip     tt = new Tooltip("Cliquez pour choisir une marque de machine");
   bouton1.setTooltip(tt);
```

Les composants(ChoiceBox)

- Les choiceBox de JavaFX permettent aux utilisateurs de faire un choix entre une liste de valeurs.

```
ChoiceBox<String> cbUFR = new ChoiceBox<String>();
cbUFR.getItems().addAll("SATIC","ECOMIJ","SDD");
cbUFR.getSelectionModel().select(0); // ou cbx cbUFR.setValue(« SATIC");
root.getChildren().add(c bUFR );
```

```
cbUFR.getSelectionModel().selectedItemProperty() .addListener((v, oldV, newV) -> {
System.out.println(oldV+", "+newV); // Display old and new values
System.out.println(cbUFR.getSelectionModel().getSelectedIndex()); });
```

Les composants(ComboBox)

- Les ComboBox sont assez similaires aux ChoiceBox et permet à l'utilisateur de choisir une valeur parmi une liste « éditable » contrairement au ChoiceBox.

```
cbRegion.getItems().addAll("Dakar", "Diourbel", "Fatick","Kaffrine", "Kaolack",
"Kedougou", "Kolda","Louga","Matam","Saint-Louis",
"Sedhiou","Tambakounda","Thies","Ziguinchor");
cbRegion.setPromptText("Choisir une region");
cbRegion.setVisibleRowCount(4);
```

```
cbRegion.setOnAction(event -> { System.out.println(cbRegion.getValue()); });
```

Les composants(Spinner)

- Le composant Spinner<T> permet à l'utilisateur de sélectionner une valeur dans une liste d'éléments grâce aux deux boutons lui permettant de défiler les valeurs.

```
Spinner<Integer> spI = new Spinner<>(-30, 30, 0, 5);
 Spinner<Double>  spD = new Spinner<>(2.0, 10.0, 4.0, 0.5);
spI.getStyleClass().add(Spinner.STYLE_CLASS_SPLIT_ARROWS_VERTICAL);
spD.getStyleClass().add(Spinner.STYLE_CLASS_SPLIT_ARROWS_HORIZONTAL);
```

```
System.out.println(spD.getValue());
```

Les composants(ListView)

- Une ListView<T> est un composant qui permet à l'utilisateur d'effectuer des sélections simples ou multiples sur les éléments affichés sur la liste.

```
listView=new ListView<String>();
ObservableList<String>
obs=FXCollections.observableArrayList("Banane","Betrave","Fraise","poire","prune","Pomme");
listView.setItems(obs);
listView.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
```

```
listView.setOnMouseClicked(event -> {
System.out.println(listView.getSelectionModel().getSelectedItems()); });
```

Les composants(TreeView)

- Le composant fournit une vue des structures hiérarchiques.
- Dans chaque arbre, l'objet le plus élevé de la hiérarchie est appelé "racine". La racine contient plusieurs éléments enfants.

```
public class ClasseTreeView extends Application{
Stage window;
TreeView<String> treeView;
TreeItem<String> root,marque,couleur;
@Override
public void start(Stage primaryStage) throws
Exception {
window=primaryStage;
window.setTitle("TreeView");
StackPane stp=new StackPane();
root=new TreeItem<>();
root.setExpanded(true);
marque= makeBranch("Marque",root);
makeBranch("Ford",marque);
makeBranch("Golf",marque);
makeBranch("Rav4",marque);
makeBranch("R19",marque);
couleur=makeBranch("Couleur", root);
makeBranch("Noire", couleur);
makeBranch("Grise", couleur);
makeBranch("Verte", couleur);
makeBranch("Bleue", couleur);
treeView=new TreeView<String>(root);
treeView.setShowRoot(false);
stp.getChildren().add(treeView);
stp.setPadding(new Insets(10));
Scene scene=new
Scene(stp,500,300,Color.GREEN);
window.setScene(scene);
window.show();}

private TreeItem<String> makeBranch(String
titre, TreeItem<String> root2) {
TreeItem<String> tItem=new TreeItem<>(titre);
tItem.setExpanded(true);
root2.getChildren().add(tItem);
return tItem;}
```

Les composants(TableView)

- Le composant le plus adapter pour représenter un tableau en javaFx est un **TableView.**

```
tableau=new TableView<Produit>();
TableColumn<Produit, String> nomCol=new TableColumn<>("Nom");
nomCol.setMinWidth(200);
nomCol.setCellValueFactory(new PropertyValueFactory<>("Nom"));
TableColumn<Produit, Double> prixCol=new TableColumn<>("Prix");
prixCol.setMinWidth(200);
prixCol.setCellValueFactory(new PropertyValueFactory<>("Prix"));
TableColumn<Produit, Integer> quantCol=new TableColumn<>("Quantite");
quantCol.setMinWidth(200);
quantCol.setCellValueFactory(new PropertyValueFactory<>("Quantite"));
tableau.setItems(getProduit());
tableau.getColumns().addAll(nomCol,prixCol,quantCol);
public ObservableList<Produit> getProduit(){
ObservableList<Produit> obProd=FXCollections.observableArrayList();
obProd.add(new Produit("Lait",10000,2));
obProd.add(new Produit("Pommade",5000,3));
obProd.add(new Produit("Lipo-levre",3000,1));
obProd.add(new Produit("Poudre-Mac",25000,2));
obProd.add(new Produit("Mascara-Mac",18000,3));
obProd.add(new Produit("Parfum",30000,2));
return obProd;

}
```

Ajouter trois zones de texte et deux boutons qui permettent d'ajouter ou de supprimer des produits.

Les composants(TableView)

```
public void ajoutProduit() {
Produit prod=new Produit();
prod.setNom(tNom.getText());
prod.setPrix(Double.parseDouble(tPrix.getText()));
prod.setQuantite(Integer.parseInt(tQuant.getText()));
tableau.getItems().add(prod);
tNom.clear();
tPrix.clear();
tQuant.clear();
}
public void supProduit() {
ObservableList<Produit> selProd,selMultProd;
selMultProd=tableau.getItems();
selProd=tableau.getSelectionModel().getSelectedItems();
selProd.forEach(selMultProd::remove);
}
```

Les Menus

Les menus déroulants usuels font intervenir plusieurs sortes d'objets:

MenuBar: Ce composant permet de créer une barre de menu

Menu: Ce composant permet de créer un menu

MenuItem: Ce composant permet de créer un **item** de **menu**

CheckMenuItem: Ce composant est un menuItem qui peut être basculé entre les états sélectionnés et désélectionnés.

RadioMenuItem: Ce composant est un menuItem avec des boutons radio

CustomMenuItem: Ce composant propose des menus personnalises, il permet de traiter n'importe quel nœud d'un graphe de scène comme un élément de menu.

SeparatorMenuItem: permet de créer un séparateur entre les menus

Contextmenu: Composant contenant un ObservableList d'éléments de menu. Le composant MenuBar n'est plus nécessaire

Les Menus

Eemple de MenuBar Menu,MenuItem, SeparatorMenuItem

```
MenuBar menubar=new MenuBar();
Menu menu=new Menu("File");
menu.getItems().add(new MenuItem("Save as..."));
menu.getItems().add(new MenuItem("Open File..."));
menu.getItems().add(new SeparatorMenuItem());
menu.getItems().add(new MenuItem("Import..."));
menu.getItems().add(new SeparatorMenuItem());
menu.getItems().add(new MenuItem("Export..."));
MenuItem minew=new MenuItem("New");
menuFile.getItems().add(minew);
```

Les Menus

Eemple de CheckMenuItem

```
Menu menuHelp=new Menu("Help");
 CheckMenuItem chmi=new CheckMenuItem("Afficher les lignes");
menuHelp.getItems().add(chmi);
```

```
chmi.setOnAction(e->{
 if(chmi.isSelected())
 System.out.println("Les lignes sont affichees");
 else
 System.out.println("Aucune ligne n'est affichee");
 });
 menuHelp.getItems().add(chmi);
```

Boite de dialogue

Une Boite de dialogue est une Interface Home Machine(IHM) qui permet:
- ❖D'informer l 'utilisateur,
- ❖d'obtenir une réponse de l'utilisateur
- ❖Ou la combinaison des deux.

Une boite de dialogue dépend d'une autre fenêtre.

Une boite de dialogue peut **être Modal ou non-Modal.**

Modal: L'utilisateur doit terminer avec la boite de dialogue avant d'interagir avec la fenêtre principale (dépendante).

non-Modal: L'utilisateur peut interagir avec les deux fenêtres en même temps. La boite est fermée explicitement par l'utilisateur.

- ➢Simple: informer, demander ou saisir une information.
- ➢Spécifique: propre à l'application

Boite d'alerte

```
public class premiereFenetre1  extends Application{
Button bouton;
public static void main(String[] args) {
// TODO Auto-generated method stub
launch(args);
}
public void start(Stage premierStage)
{
premierStage.setTitle("Ma premiere application javaFx");
bouton=new Button("confirmer");
StackPane root=new StackPane();
root.getChildren().addAll(bouton,bouton1);
Scene scene=new Scene(root,300,250);
bouton.setOnAction(e->BoiteAlert.display("Boite d'Alerte"," Attention !!!"));
premierStage.setScene(scene);
premierStage.show();}}
```

Boite d'alerte

```
public class BoiteAlert {
public static void display(String titre,String
message){
Stage fenetre=new Stage();
fenetre.initModality(Modality.APPLICATION_MO
DAL);
fenetre.setTitle(titre);
fenetre.setMaxWidth(250);
Label label1=new Label();
label1.setText(message);
Button fermer =new Button("Fermer la
fenetre");
fermer.setOnAction(e->fenetre.close());
VBox vb=new VBox(10);
vb.getChildren().addAll(label1,fermer);
vb.setAlignment(Pos.CENTER);
Scene scene=new Scene(vb);
fenetre.setScene(scene);
fenetre.showAndWait();}}
```

Boite de dialogue Alerte (Information)

```
public class BoiteInformation {
 public static void information(){
 Alert dialog=new Alert(AlertType.INFORMATION);
 dialog.setHeaderText("Information importante");
 dialog.setTitle("Boite d'information");
 dialog.setContentText("Travaux pratiques pour la
semaine prochaine!!!!\n"+
 "Bonne chance!!!!");
 dialog.showAndWait(); }}
```

Boite de dialogue Alerte(Warning)

```
public class BoiteAlerte {
public static void alerter(){
Alert alerte = new Alert(AlertType.WARNING);
alerte.setTitle("Boite d'Alerte");
alerte.setHeaderText(null);

alerte.setContentText("Bourse : Il ne vous reste que 2000fCFA avant la fin du mois !");
alerte.showAndWait();
}}
```

Boite de dialogue Alerte(Erreur)

```
public class BoiteErreur {
public static void erreur(){
Alert err = new Alert(AlertType.ERROR);
err.setTitle("Boite d'Alerte Erreur ");
err.setHeaderText("impossible d'ecrire sur le fichier");
err.setContentText("Error : chemin du specifie introuvable !");
err.showAndWait();
}}
```

Boite de dialogue Alerte(Confirmation)

```
public class BoiteConfirmation {
public static void confirmer(){
Alert confirme = new Alert(AlertType.CONFIRMATION);
confirme.setTitle("Boite de Confirmation");
confirme.setHeaderText(null);
confirme.setContentText("Votre ordinateur va redemarrer dans 5mn");
Optional<ButtonType> reponse = confirme.showAndWait();
if (reponse.get() == ButtonType.OK) {
System.out.println("l'utilisateur veut redemarrer"); }
else { System.out.println("l'utilisateur ne veut pas redemarrer"); }}}
```

Boite de dialogue Alerte(confirmation personnalisee)

```
public class BoiteConfirmationPersonnalisee {
public static confirmationPersonnalisee(){
Alert confPers = new Alert(AlertType.CONFIRMATION);
confPers.setTitle("Boite de confirmation personnalisee");
confPers.setHeaderText("Ordinateur : Les meilleurs  marques !");
confPers.setContentText("Choisissez votre marque:");
ButtonType btnSBP = new ButtonType("Surface  book pro");
ButtonType btnHPS = new ButtonType("HP spectre");
ButtonType btnMBP = new ButtonType("Mac  book pro");
ButtonType btnCancel = new ButtonType("Cancel", ButtonData.CANCEL_CLOSE);
confPers.getButtonTypes().setAll(btnSBP, btnHPS, btnMBP, btnCancel);
Optional<ButtonType> choice = confPers.showAndWait();
if (choice.get() == btnSBP) { System.out.println ("Ordinateur: Surfacebook pro"); }
else if (choice.get() == btnHPS) {System.out.println ("Ordinateur: HP spectre");}
else if (choice.get() == btnMBP) {System.out.println ("Ordinateur: Macbook pro");}}
else {System.out.println ("fermer ou annuler");}}}}
```

Boite de dialogue Texte (TextInputDialog)

```
public class LigneDeTexte{
public static void ligneText(){
TextInputDialog tid = new TextInputDialog("saisir le nom");
tid.setTitle("Une ligne de texte");
tid.setHeaderText("Information");
tid.setContentText("Prenom :");
Optional<String> textIn = tid.showAndWait();
if (textIn.isPresent()) { System.out.println("Nom utilisateur = " + textIn.get()); }
}
}
```

Boite de dialogue avec choix(ChoiceDialog)

```
public class BoiteDechoix {
public static void choix(){
String[]  choices = {"SurfaceBook pro", "HP spectre",
"Macbook pro", "DELL"};
ChoiceDialog<String> cDial = new
ChoiceDialog<>(choices[1], choices);
cDial.setTitle("Choix");
cDial.setHeaderText("Selectionner ordinateur:");
cDial.setContentText("Marque:");
Optional<String> selection = cDial.showAndWait();
selection.ifPresent(str ->
System.out.println("Selection:" + str));
}}
```

Explorateur Window(FileChooser)

FileChooser permet de choisir un ou plusieurs fichiers dans l'explorateur Window.
On utilise **DirectoryChooser** pour un repertoire.

showOpenDialog()	Sélectionner un fichier
showOpenMultipleDialog()	Sélectionner plusieurs fichiers
showSaveDialog()	Sauvegarder un fichier

Explorateur Window(FileChooser)

```
public class ChoisirUnFichier {
static FileChooser fileChooser;
public static void choisirFichier(){
 fileChooser = new FileChooser();
fileChooser.setTitle(« Choisir un fichier");
File homeDir = new File(System.getProperty("user.home"));
fileChooser.setInitialDirectory(homeDir);
fileChooser.getExtensionFilters().addAll(
new ExtensionFilter("Text Files" , "*.txt"),
new ExtensionFilter("Image Files", "*.png", "*.jpg", "*.gif"),
new ExtensionFilter("Audio Files", "*.wav", "*.mp3", "*.aac"),
new ExtensionFilter(« Tous les fichiers" , "*.*"));}}
```

```
 File selectedFile = ChoisirUnFichier.fileChooser.showOpenDialog(premierStage);
if (selectedFile != null) { try {
Desktop.getDesktop().open(selectedFile); }
catch (Exception a) { System.err.println(“Erreur d’ouverture du fichier"); } });.
```

Explorateur Window(FileChooser)

Pour sélectionner plusieurs fichiers on utilise showOpenMultipleDialog(Stage)

```
List<File> selectedFiles =
ChoisirUnFichier.fileChooser.showOpenMultipleDialog(premierStage);
if (selectedFiles != null) { for (File f : selectedFiles)  System.out.println(f); }});
```

DirectoryChooser est utilisée pour sélectionner un répertoire.

```
DirectoryChooser dirChooser = new DirectoryChooser();
dirChooser.setTitle("Choisir un repertoire");
dirChooser.setInitialDirectory(new File("D://DevJavaF"));

File selectedFile = dirChooser.showDialog(premierStage);
 if (selectedFile != null) { try { Desktop.getDesktop().open(selectedFile); }
catch (Exception a) { a.printStackTrace(); } }
```

DatePicker

DatePicker permet à l'utilisateur de choisir une date dans un calendrier.

Elle se présente sous forme de comboBox éditable

```
DatePicker dPicker = new DatePicker();
hb.getChildren().add(dPicker);
dPicker.setOnAction(e -> { LocalDate ldate = dPicker.getValue();
System.out.println("La date: " + ldate);
});
```

ColorPicker

ColorPicker est un composant qui permet à l'utilisateur de choisir une couleur dans une palette de couleurs . Tout comme les **DatePicker**, elle se présente sous forme de comboBox éditable

```
ColorPicker cPicker = new ColorPicker(Color.BLUE);
hb.getChildren().add(cPicker);
```

JavaFX avec CSS

Avec JavaFX vous pouvez appliquer un style en utilisant les Cascading Style Sheets (CSS). Il est maintenant plus facile de personnaliser l'apparence d'une application Java.

```
.root{
-fx-background-color: greenyellow;
-fx-font-size: 11pt;
}
.label{
-fx-text-fill: black ;
}
.button{
-fx-background-color: green ;
-fx-text-fill: #FFFFFF;
-fx-background-radius: 8;
}
```

On peut ajouter **linear-gradient pour avoir plusieurs couleurs.**

```
scene.getStylesheets().add("application.css");
```

Introduction a FXML

Le FXML est un format de fichier propre à JavaFX et servant, entre autres, à définir des interfaces graphiques. Il utilise une syntaxe XML.

```
public class Main extends Application {
@Override
public void start(Stage primaryStage) {
try {
VBox root =
(VBox)FXMLLoader.load(getClass().getResource(
"Sample.fxml"));
Scene scene = new Scene(root,400,400);
//scene.getStylesheets().add(getClass().getResource(
"application.css").toExternalForm());
primaryStage.setScene(scene);
primaryStage.show();
} catch(Exception e) {
e.printStackTrace();}}

public class SampleController {
public Button bouton;
public void handleButtonClick(){
System.out.println(" Reagir avec un
controleur");}}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<VBox xmlns:fx="http://javafx.com/fxml/1" xmlns="http://javafx.com/javafx/8"
fx:controller="application.SampleController">
  <Label text="J'aime javafx"/>
  <Button text="Valider" fx:id="bouton" onAction="#handleButtonClick"/>
</VBox>
```

Graphique(Chart: Circulaire)

Graphique: Diagramme circulaire.

```
root=new BorderPane();

        pieChartData =
FXCollections.observableArrayList(
         new PieChart.Data("Sapotis", 13),
         new PieChart.Data("Oranges", 25),
         new PieChart.Data("Citrons", 10),
         new PieChart.Data("Mangues", 22),
         new PieChart.Data("Pasteques", 30));
chart = new PieChart(pieChartData);
chart.setTitle("Fruits locaux");
```

Legende:

```
chart.setLabelLineLength(10);
chart.setLegendSide(Side.LEFT);
```

Evenement:

```
chart.getData().forEach((data) -> {
        data.getNode().addEventHandler(MouseEvent.MOUSE_PRESSED, (MouseEvent e) -
> {
            lb.setTranslateX(e.getSceneX());
            lb.setTranslateY(e.getSceneY());
            lb.setText(String.valueOf(data.getPieValue()) + "%");
        }); });
```

Graphique(Chart: ligne)

Graphique: Courbe.

```
xAbs = new NumberAxis();
yOrd = new NumberAxis();
xAbs.setLabel("Nombre de mois");
lineChart=new LineChart<>(xAbs,yOrd);
lineChart.setTitle("Quantité en stock, 2019");
series = new XYChart.Series();
 series.setName("Quantité en stock Annuel");
 series.getData().add(new XYChart.Data(1, 23));
 series.getData().add(new XYChart.Data(2, 14));
 series.getData().add(new XYChart.Data(3, 15));
 series.getData().add(new XYChart.Data(4, 24));
series.getData().add(new XYChart.Data(5, 34));
series.getData().add(new XYChart.Data(6, 36));
 series.getData().add(new XYChart.Data(7, 22));
 series.getData().add(new XYChart.Data(8, 45));
 series.getData().add(new XYChart.Data(9, 43));
 series.getData().add(new XYChart.Data(10, 17));
 series.getData().add(new XYChart.Data(11, 29));
 series.getData().add(new XYChart.Data(12, 25));
 lineChart.getData().add(series);
```

Graphique(Chart: ligne)

Graphique: Zone.

```
xAbs = new NumberAxis();
yOrd = new NumberAxis();
xAbs.setLabel("Nombre de mois");
lineChart=new LineChart<>(xAbs,yOrd);
lineChart.setTitle("Quantité en stock, 2019");
series = new XYChart.Series();
 series.setName("Quantité en stock Annuel");
 series.getData().add(new XYChart.Data(1, 23));
 series.getData().add(new XYChart.Data(2, 14));
 series.getData().add(new XYChart.Data(3, 15));
 series.getData().add(new XYChart.Data(4, 24));
series.getData().add(new XYChart.Data(5, 34));
series.getData().add(new XYChart.Data(6, 36));
 series.getData().add(new XYChart.Data(7, 22));
 series.getData().add(new XYChart.Data(8, 45));
 series.getData().add(new XYChart.Data(9, 43));
 series.getData().add(new XYChart.Data(10, 17));
 series.getData().add(new XYChart.Data(11, 29));
 series.getData().add(new XYChart.Data(12, 25));
 lineChart.getData().add(series);
```

Graphe 3D

Sphere et cube.

```
sphere = new Sphere(2.5);
        pm=new PhongMaterial(Color.FORESTGREEN);
        box=new Box(5, 5, 5);
        sphere.setMaterial(pm);
        sphere.setTranslateZ(7);
        sphere.setTranslateX(2);
        root=new Group();
        box.setMaterial(new
 PhongMaterial(Color.RED));
 pivot=new Translate();
   rotY= new Rotate(0, Rotate.Y_AXIS);
  // Position de la camera
    camera = new PerspectiveCamera(true);
camera.getTransforms().addAll ( pivot, rotY,
                new Rotate(-20, Rotate.X_AXIS),
                new Translate(0,  0, -50) );

timeline  = new Timeline(    new KeyFrame(
Duration.seconds(0),  new KeyValue(rotY.angleProperty(),  0)
), new KeyFrame(Duration.seconds(15),
 new KeyValue(rotY.angleProperty(),  360) ) );

timeline.setCycleCount(Timeline.INDEFINITE);
timeline.play();
root.getChildren().addAll(box,sphere);
root.getChildren().stream()
.filter(node -> !(node instanceof Camera))
```

```
Sphere .forEach(node ->
                    node.setOnMouseClicked(event -> {
                        pivot.setX(node.getTranslateX());
                        pivot.setY(node.getTranslateY());
                        pivot.setZ(node.getTranslateZ());
                    }) );
scene=new Scene(root,300,300,true, SceneAntialiasing.BALANCED  );
        scene.setFill(Color.ALICEBLUE);
        scene.setCamera(camera);
        stage.setScene(scene);
        stage.setResizable(false);
        stage.show();    }
```

CHAPITRE VIII

JDBC

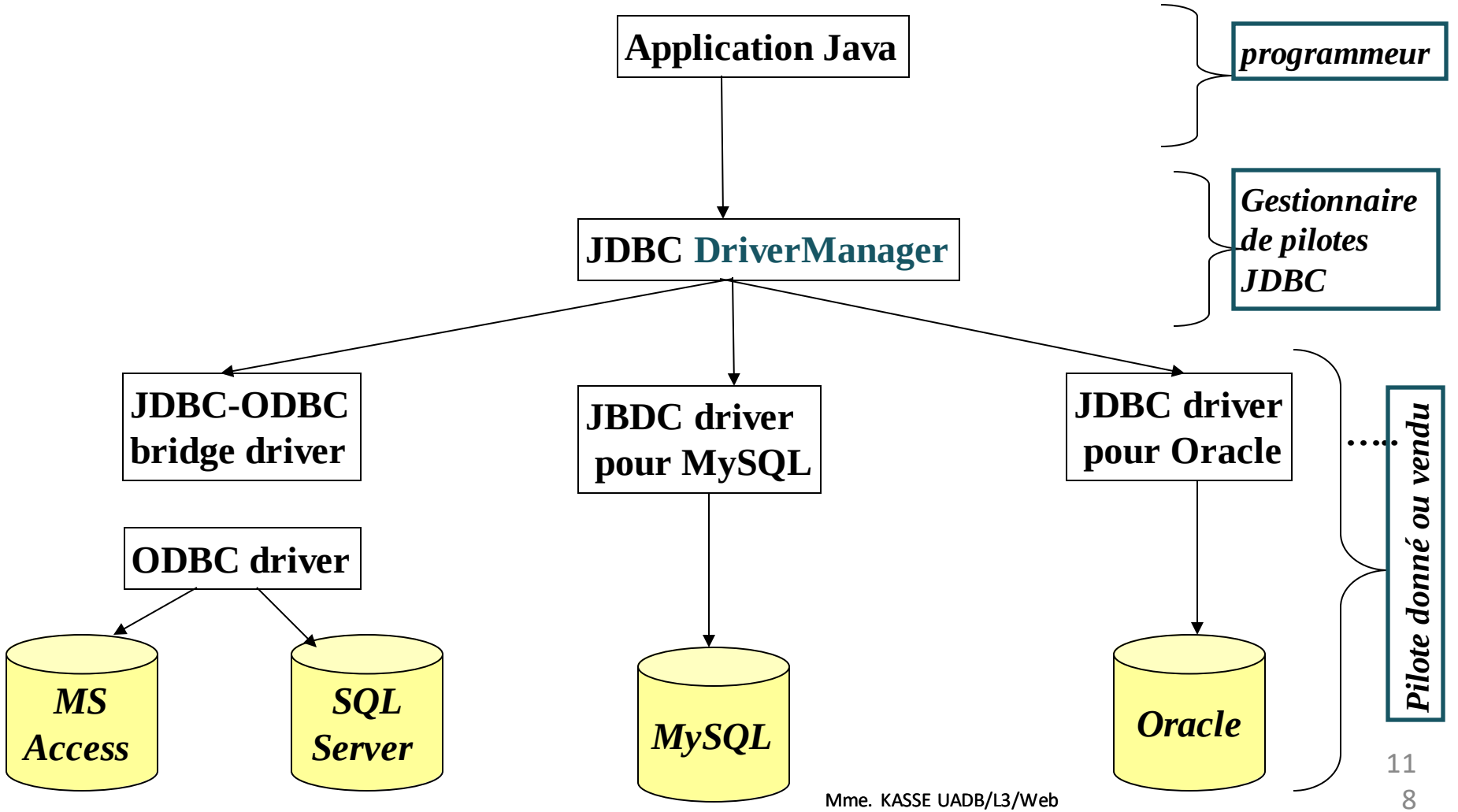
JDBC(Java DataBase Connectivity)

- JDBC permet d'accéder aux bases de données à l'aide
- du langage Java via des requêtes SQL (langage permettant de dialoguer avec un SGBDR).
- JDBC est fourni par le paquetage java.sql

- Dans une application JAVA utilisant une base de données avec l'interface JDBC, on trouvera généralement les étapes suivantes :

- 1. Connexion à la base de données
- 2. Émission de requêtes SQL vers la base
- 3. Réception et traitement des résultats de ces requêtes
- 4. Fermeture de la connexion

JDBC(Java DataBase Connectivity)



JDBC(Java DataBase Connectivity)

- Pour travailler avec un SGBD, il faut disposer de classes (driver) qui implémentent les interfaces de JDBC.

- JDBC est totalement indépendant de tout SGBD: la même application peut être utilisée pour accéder à une base Oracle, Sybase, MySQL,etc.

• Chargement du pilote

- Un programme JDBC débute toujours par le chargement du pilote approprié pour la BD.
- Mais puisque le programme a la possibilité d'accéder à plusieurs types de BD, il peut avoir plusieurs pilotes.

- C'est au moment de la connexion à la BD que le DriverManager choisit alors le *bon* pilote.

- DriverManager = gestionnaire de tous les drivers chargés par un programme java

URLs : JDBC(Java DataBase Connectivity)

Après le chargement du driver, il faut localiser la BD en spécifiant son emplacement.
Pour chaque type de driver il existe un schéma de désignation de la base à atteindre avec JDBC:

Chaque URL est de la forme: jdbc: sous-protocole :base_de_donnée

Structure d'un programme: JDBC(Java DataBase Connectivity)

1. Chargement du driver (= chargement de la classe du driver dans la JVM)

```
Class.forName (String driverName);
        Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver") ;
        Class.forName ("oracle.jdbc.driver.OracleDriver") ;
        Class.forName ("org.gjt.mm.mysql.Driver") ;
```

Quand une classe *Driver* est chargée, elle doit créer une instance d'elle même et s'enregistrer auprès du *DriverManager*.

Certains compilateurs refusent la notation précédente et demandent :

- Class.forName (" driver_name").newInstance ( ) ;
- Cette étape 1 constitue *l'enregistrement* du driver JDBC.

2. Établir une connexion à la base de données

Une fois le driver enregistré, une connexion peut être établie avec la BD.

pour obtenir une connexion à un SGBD, il faut faire une demande à la classe gestionnaire de drivers:

Demande permise par la méthode **getConnection (…)** de la classe **DriverManager**

Cette méthode peut prendre 3 arguments au plus:

- *l'URL vers la BD*
- *le nom de l'utilisateur de la base*
- *son mot de passe*

cette méthode renvoie un objet de type **Connection**.

```
Connection con =
  DriverManager.getConnection ("jdbc:odbc:MasterI", "login", "passwd");
```

Structure d'un programme: JDBC(Java DataBase Connectivity)

3. Création de requêtes SQL en utilisant l'objet de type Connection (étape 2).

dans cette étape on crée en fait une zone de description de requêtes ie un espace où l'on pourra exécuter des opérations SQL.
cette zone est un objet de la classe **Statement** que l'on crée par la méthode **createStatement ( ) .**

```
Statement st = con.createStatement ( ) ;
```

Structure d'un programme: JDBC(Java DataBase Connectivity)

4. Envoi et exécution de requêtes.

Il existe trois types d'exécution de requêtes:

➢ **executeQuery (…):** pour les requêtes SELECT qui retournent un **ResultSet** (tuples),

➢ **executeUpdate (…):** pour les requêtes INSERT, UPDATE, DELETE, CREATE TABLE DROP TABLE qui retourne un entier **(int)** désignant le nombre de tuples traités.

➢ **execute ( ):** pour les procédures stockées (cas rares).

Les méthodes **executeQuery ( )** et **executeUpdate( )** de la classe **Statement** prennent comme argument une chaîne **(String)** indiquant la requête SQL à exécuter.

```
Statement st = con.createStatement ( );
ResultSet rs = st.executeQuery ("select * from Client") ;
```

Traitement des données retournées JDBC(Java DataBase Connectivity)

- si une requête ne peut s'exécuter ou qu'une erreur de syntaxe SQL a été
- détectée, l'exception **SQLException** est levée.
- Le driver JDBC effectue d'abord un premier accès à la BD pour découvrir les types des colonnes impliquées dans la requête puis un deuxième pour l'exécuter.

La méthode **next ( )** de **ResultSet** permet de parcourir itérativement ligne par ligne l'ensemble des tuples sélectionnés.
Cette méthode :
- retourne **false** si le dernier tuple est lu, **true** sinon,
- chaque appel fait avancer le curseur sur le tuple suivant,
- initialement, le curseur est positionné avant le premier tuple

Exécuter **next ( )** au moins une fois pour avoir le premier.

```
while (rs.next ( )) { //traitement tuple par tuple}
```

Traitement des données retournées JDBC(Java DataBase Connectivity)

Les *colonnes* d'une table de la BD sont référencées par **leur numéro** (commençant par 1) ou par leur **nom.**
L'accès aux valeurs des colonnes se fait par des méthodes de la forme **getXXX (…)** permettant la lecture de données du type **XXX** dans chaque colonne du tuple courant.

```
int val = rs.getInt (3) ; // acces  à la 3e colonne

String prod = rs.getString ("Produit") ;// acces à la colonne de nom Produit
```

Utilisation de JDBC pour MySQL

```
import java.sql .*;
public class TestJavaMySQL {
  static Connection con;
  static Statement st;
  static ResultSet rs;
  public static void main (String[] args) {

  try {  /*chargement du driver*/
          Class.forName("com.mysql.jdbc.Driver").newInstance ( )  ;
        }
  catch (Exception er_dv){System.out .println("Erreur driver:  "+er_dv.getMessage() ) ;}
```

Utilisation de JDBC pour MySQL

```
try {con = DriverManager.getConnection ("jdbc:mysql: //localhost /MasterI","root","") ;}
 catch (Exception er_con ){System.out.println("Erreur de connexion "+ er_con.getMessage (
));}
 try {  st = con.createStatement() ; }
 catch (SQLException er_st){System.out.println ("Erreur de Statement "+er_st.getMessage());}
 try {
       rs = st.executeQuery("select * from client") ;
          while (rs.next() )
            { System.out .println(rs.getObject (1)+" "+rs.getObject(2)+" "+rs.getObject(3) ) ;}}
 catch (Exception er_rs)  {System.out .println("Erreur ResultSet "+er_rs.getMessage ( ) ); }
  try { rs.close ( ) ;       st.close ( );       con.close ( );}
  catch  (Exception d) { }
 }}
```

Correspondances SQL/JAVA

Si vous écrivez une instruction telle que String prenom=rs.getString(" prenom");
cela signifie que vous êtes convaincu que la colonne de nom PRENOM a été créée sous SQL avec le type *VARCHAR*. Autrement cette instruction causerais une erreur de runtime.
Donc il est bon de savoir la correspondance entre types SQL et types Java pour pouvoir manipuler convenablement les données lues.

Types SQL	Types Java	Méthodes
CHAR/ VARCHAR	String	getString ( )
INTEGER	int	getInt ( )
TINYINT	byte	getByte ( )
SMALLINT	short	getShort ( )

Correspondances SQL/JAVA

Types SQL	Types Java	Méthodes
BIGINT	long	getLong ( )
bit	boolean	getBoolean ( )
REAL	float	getFloat ( )
NUMERIC DECIMAL	java.math.BigDecimal	getBigDecimal ( )
DATE	java.sql.Date	getDate ( )
TIME	java.sql.Time	getTime ( )
TIMESTAMP	java.sql.TimeStamp	getTimeStamp ( )
FLOAT DOUBLE	double	getDouble ( )

Les MétaDonnées

On peut avoir besoin quelque fois des informations sur la structure et le schéma de la base de données elle-même. Ces informations sont appelées des Métadonnées.
Pour obtenir ces renseignements, il faut utiliser un objet de la classe DatabaseMetaData à partir de l'objet Connection. La méthode getMetaData ( ) de la classe Connection permet de créer cet objet.

```
DatabaseMetaData donneesBD = objet_con.getMetaData ( ) ;
```

A partir de cet objet, cette classe fournit beaucoup de renseignements sur la BD.
Par exemple:

```
DatabaseMetaData d = con.getMetaData ( ) ;
String nomSGBD       = d.getDatabaseProductName ( ) ;
String versionDriver = d.getDriverVersion ( );
String nomDriver     = d.getDriverName ( );
```

Les MétaDonnées

On peut aussi avoir des informations sur les objets ResultSet générés:
- nombre de colonnes contenues dans le ResultSet,
- noms des colonnes
- types des colonnes,
- etc.

Pour récupérer ces Métadonnées, il faut créer un objet de la classe ResultSetMetaData grâce à la méthode getMetaData ( ) de la classe ResultSet.

On notera deux méthodes utiles dans la classe *ResultSetMetaData* :
1. int getColumnCount() qui donne le nombre de colonnes du *ResultSet*
2. String getColumnLabel(int i) qui donne le nom de la colonne i du *ResultSet* (i>=1)

```
ResultSetMetaData rsmd = objet_resultset.getMetaData ( );

int columnCount = rsmd.getColumnCount ( );
String columnLabel = rsmd.getColumnLabel ( i );
String columnType = rsmd.getColumnTypeName ( i ) ;
```

References:
https://fsznajderman.wordpress.com/2011/03/19/la-covariance-en-java/

http://programmation-java.1sur1.com/Java/Tutoriels/Tiger5.0/genericite.php

http://en.wikipedia.org/wiki/Covariance_and_contravariance_(computer_science)

Name: e(fx)clipse
Location: http://download.eclipse.org/efxclipse/updates-released/3.0.0/site